GreenStock Design Documentation

Name: Varshini M

1. Executive Summary

GreenStock is a sustainability-focused inventory management system designed to track perishable items, monitor expiry dates, and reduce waste through intelligent forecasting and alerts. The system enables quick data entry and operates offline. AI-powered insights are provided with graceful fallback mechanisms, ensuring no hard dependencies on external services.

Core Philosophy: Simplicity over complexity, offline-first, AI-enhanced not AI-dependent

---

2. System Architecture

2.1 High-Level Overview

The application follows a client-only architecture with React as the frontend framework. All data persistence occurs in the browser's localStorage, making the application fully functional offline without any backend dependencies. External services for AI analysis and OCR are optional and implemented with graceful fallback mechanisms.

React Frontend (SPA) (React 18 + TypeScript + Vite)
State Management • useInventory Hook (custom) • inventoryStore (localStorage-based) • React Query (for optional API calls)
Business Logic Layer • forecasting.ts (AI suggestions) • aiAnalysis.ts (Gemini/OpenAI integration) • imageOCR.ts (Tesseract.js)
Data Persistence • localStorage (primary - 5-50MB per domain) • seed-data.json (auto-seeded on first run) • [FUTURE] Cloud sync (Firebase/Supabase)
External Services • Google Gemini API • [FUTURE] Barcode services

2.2 Data Flow

User input flows through a validation layer using Zod schemas, then passes to custom hooks that manage CRUD operations. Data is serialized to JSON and stored in localStorage. The forecasting engine analyzes consumption patterns to generate reorder suggestions, which are displayed on the dashboard with Recharts visualizations.

2.3 Offline-First Architecture

The application prioritizes offline functionality:

- Primary Storage: Browser localStorage (synchronous, always available)

- Fallback Handling: Manual thresholds activate when AI services unavailable

- Future Sync Strategy: Periodic background synchronization with cloud

- Conflict Resolution: Last-write-wins with timestamp tracking

---

3. Technical Stack

3.1 Frontend Framework

- React 18.3.1: UI component library

- TypeScript 5.8.3: Static type checking

- Vite 5.4.19: Build tool and development server

- SWC 3.11.0: Fast JavaScript/TypeScript compiler

3.2 Styling and UI

- Tailwind CSS 3.4.17: Utility-first CSS framework

- Shadcn/ui: Accessible component library built on Radix UI

- Lucide React 0.462.0: Icon set

- Tailwind Animate 1.0.7: CSS animation utilities

3.3 Forms and Validation

- React Hook Form 7.61.1: Form state management

- Zod 3.25.7: Runtime schema validation

- @hookform/resolvers 3.10. : Zod integration with React Hook Form

3.4 Routing and Navigation

- React Router 6.30.1: Client-side routing

- Resizable Panels 2.1.9: Collapsible sidebar navigation

3.5 Data and Collections

- React Query 5.83.0: Server state management (future backend)

- Recharts 2.15.4: Data visualization

- date-fns 3.6.0: Date manipulation

3.6 AI and Image Processing

- Google Gemini API: Primary AI trend analysis

- Tesseract.js 7.0.0: OCR engine for text extraction

- Custom Algorithms: Moving average forecasting

3.7 Testing and Quality

- Vitest 3.2.4: Unit testing

- React Testing Library 16.0.0:Component testing

- Playwright 1.57.0: End-to-end testing

- ESLint 9.32.0: Code linting

- PostCSS 8.5.6: CSS processing

3.8 State and Notifications

- Sonner 1.7.4: Toast notifications

---

4. Core Design Choices

4.1 Offline-First Storage with localStorage

Decision Rationale: Browser localStorage was chosen as the primary data store to eliminate backend dependencies and enable immediate offline functionality.

Advantages:

- Zero backend setup required

- Complete offline functionality

- Synchronous access (no loading states)

- 5-50MB capacity sufficient for <1,000 items

- Familiar implementation for all developers

Limitations:

- Single-origin policy prevents cross-domain data access

- Manual synchronization required for multi-device access

- No real-time collaboration

- Limited audit trail (timestamps only)

4.2 AI with Graceful Fallback

Decision Rationale: AI functionality is implemented as enhancement rather than a core requirement to ensure the application remains functional without external dependencies.

AI Stack:

-Primary: Moving average forecasting (no external dependencies)

- Secondary: Gemini/OpenAI API (requires API key)

- Fallback: Manual min/max threshold alerts (always functional)

Benefits:

- Immediate usability without API setup

- Dashboard toggle between AI and Manual modes

- Automatic fallback when APIs fail

- Forecasting works with sparse data (2-3 consumption records)

4.3 Moving Average Forecasting Algorithm

Decision Rationale: A 30-day moving average was selected over complex ML models for its simplicity, interpretability, time limiationand effectiveness with limited data.

Algorithm Steps:

1. Retrieve consumption records from last 30 days

2. Calculate total quantity consumed

3. Divide by actual time span (accounting for data gaps)

4. Compute daily usage rate

5. Calculate days until depletion: current quantity / daily rate

6. Suggest reorder when <14 days remaining

7. Calculate order quantity: daily rate × (lead time + buffer + 7 days)

4.4 Component-Driven UI with Shadcn/ui

Decision Rationale: Shadcn/ui provides pre-built accessible components with Tailwind CSS customization, offering the perfect balance between development speed and design flexibility.

Advantages:

- Accessible components out of the box

- Highly customizable via Tailwind

- Copy-paste approach eliminates dependency lock-in

- Built-in dark mode support

- Mobile-responsive default styling

4.5 TypeScript for Type Safety

Decision Rationale: Strict TypeScript implementation ensures type safety across the entire codebase, reducing runtime errors and improving developer experience.

Type Coverage:

- All data models typed

- API responses typed

- Event handlers typed

- Form validation integrated with Zod types

4.6 Zod for Runtime Validation

Decision Rationale: Zod provides type-safe schemas with human-readable error messages, ensuring consistent validation across client and future server implementations.

4.7 Mobile-First Responsive Design

Decision Rationale: Building mobile-first ensures the application is accessible on all devices, with progressive enhancement for larger screens.

4.8 Comprehensive Testing Strategy

Decision Rationale: A multi-layered testing approach ensures reliability of critical paths while maintaining development velocity.

Testing Pyramid:

- E2E (Playwright): Critical user flows

- Integration (React Testing Library): Component interactions

- Unit (Vitest): Algorithms and utilities

Coverage Areas:

- Forecasting algorithm (10+ test cases)

- Edge cases (empty data, past dates, well-stocked items)

- Form validation

- CRUD operations

- End-to-end workflows

---

5. Data Models

5.1 Core Entities

Item (Inventory Item)

```
id: string;                 // UUID
name: string;               // 1-100 characters
quantity: number;           // current stock (≥0)
expiryDate: string;         // ISO-8601 (future date)
category: "produce" | "dairy" | "meat" | "pantry" | "other";
location: string;           // storage location
minThreshold: number;       // reorder point
maxThreshold: number;       // maximum stock level
```

```
  unit: string;                // kg, pieces, liters, etc.
  createdAt: string;           // ISO-8601
  updatedAt: string;           // ISO-8601
  soldOutDate?: string;        // ISO-8601 (when quantity hit zero)
```

Constraints:

- `quantity` must be ≥ 0
- `minThreshold` must be < `maxThreshold`
- `expiryDate` must be in the future
- `name` must be between 1-100 characters

ConsumptionRecord

```
  id: string;                  // UUID
  itemId: string;              // References Item
  quantity: number;            // Amount consumed (positive) or added (negative)
  date: string;                // ISO-8601
  reason: "used" | "wasted" | "donated";
```

Purpose:

- Track inventory movement
- Calculate daily usage rates
- Analyze waste patterns
- Support emergency donation tracking

ActivityLog

```
  id: string;                  // UUID
  action: "created" | "updated" | "deleted" | "quantity_changed" | "wasted";
  itemName: string;            // Human-readable item name
  details: string;             // Description of the action
  timestamp: string;           // ISO-8601
```

Purpose:

- Provide audit trail

- Debug inventory changes

- Support compliance requirements

- Enable user education

5.2 Storage Keys

All data stored in localStorage with prefixed keys:

- `greentech_items`: Item[]

- `greentech_consumption`: ConsumptionRecord[]

- `greentech_activity`: ActivityLog[]

- `greentech_mode`: "ai" | "manual"

- `greentech_seeded`: boolean (first-run flag)

---

6. Component Architecture

6.1 AppLayout.tsx (Root Layout)

The root layout provides the application shell with responsive navigation:

- Header: Logo, theme toggle, user menu (future)

- Sidebar: Navigation links (collapsible on mobile)

- Main Content: Page router outlet

- Toast Container: Sonner notifications

6.2 DashboardPage.tsx

The dashboard serves as the home page with critical information:

Sections:

- Critical Alerts: Expiring items, low stock, reorder suggestions

- Quick Actions: Add item, bulk import, AI/manual toggle

- Quick Stats: Total items, stock quantity, waste percentage

- Consumption Charts: Daily trends, category breakdown, waste analysis

6.3 InventoryPage.tsx

The inventory page provides comprehensive item management:

Features:

- Search and filter bar

- Category and status filters

- Sortable table columns

- Expiry status color coding

- CRUD operations with confirmation

Expiry Status Colors:

- 🔴 Expired: Red/Crimson (past date)

- 🔴 Critical: Orange-Red (< 3 days)

- 🟡 Warning: Yellow (3-7 days)

- 🟢 Good: Green (> 7 days)

6.4 ItemForm.tsx

The item form handles create and update operations:

Fields:

- Name (required)

- Category (dropdown, required)

- Quantity and Unit (required)

- Expiry Date and Location

- Min/Max Thresholds

Validation Features:

- Real-time field validation

- Visual feedback for errors

- Submit disabled until valid

- Success/error toast notifications

6.5 BulkImportDialog.tsx

The bulk import feature supports CSV uploads:

Format:

name,quantity,category,expiryDate,unit,location,minThreshold,maxThreshold

Milk,5,dairy,2026-03-25,liters,Fridge,2,8

Bread,2,pantry,2026-03-22,pieces,Counter,1,4

---

7. Future Enhancements

1. Voiced based Inventory updates

2. Bar code scanning for inventory updates

3. Backend for multi-user collaboration and for role based access.

4. Advanced analytics & visualization dashboard

5. Personalized ML models trained on historical data for analysis , summarization, reorder plan

6. Improve UI/UX

7. Cloud storage for backup which syncs periodically with local storage